KR Project 2: Argumentation

December 14, 2023

Abstract

This project explores the dynamics of argumentation within intelligent decision-making systems through a blend of theoretical and computational approaches. The study introduces a preferred discussion game using Dung's abstract argumentation framework. An additional implementation includes an argumentation solver exploring credulous acceptability of arguments under admissible, preferred, and complete semantics. The results of the study provide a successful integration of a theoretical basis into functional implementations Offering perspectives on argumentation and the acceptability of arguments under different semantic conditions.

1 Introduction

Knowledge representation serves as a fundamental part in developing intelligent decision-making systems and handling complex reasoning tasks [1]. Robust formalisms are used for the modeling and assessment of argumentations central to these decision-systems. One widely used formalism is Dung's abstract argumentation framework [2]. This framework consists of arguments (A) and binary relationships (R) between these arguments that capture notions of support and conflict. The framework is used to reject or accept argumentations in a knowledge base based on defined attack (conflict) relationships between arguments.

Different types of semantics are used to determine the acceptability of an argument within an abstract argumentation framework. Among these, admissible semantics is particularly significant in characterizing sets of acceptable arguments [3]. In admissible sets we only obtain arguments where the argument is either directly accepted or defended against attacks of other arguments. A subset of the admissible semantics is the preferred semantics, which is the maximal admissible semantics. In this case of maximality, no additional arguments can be added to the preferred set without compromising admissibility. Examining both admissibility and maximality has been demonstrated to be computationally complex [4]. Consequently, decision problems involving preferred semantics can pose greater challenges in terms of solution complexity [5].

Since preferred semantics capture important properties in argumentation scenarios, this project models a preferred discussion game trying to encapsulate the concept of preferred semantics. The preferred discussion game consists of a proponent defending arguments and an opponent challenging arguments. The specific argument needs to be successfully defended to be credulously accepted under preferred semantics.

As an additional exploration of the reasoning process in argumentation frameworks, this project creates a model that explores credulous acceptance of arguments under three different semantics. Admissible semantics, preferred semantics, and complete semantics are incorporated in this model which enbody distinct principles for accepting or rejecting arguments. This model gives insight into the difference in acceptance among the different semantics, contributing to a deeper understanding of these distinct reasoning principles.

This project aims to capture the difference in semantics employed within abstract argumentation frameworks. Consequently, we try to provide insights into the mechanisms of these semantics with a few examples, offering practical knowledge to inform the selection of semantics for specific applications.

2 Implementation

Overall, the project's implementation defines a discussion game framework and an argumentation framework that simulate debates on various topics, such as work-life balance, education vs. work

for skills, and work from home vs. office. Arguments serve to express diverse viewpoints, reflecting perspectives on aspects such as the significance of completing work projects, the role of breaks and vacations in well-being, the balance between work and personal projects, the impact of consistent work efforts on professional success, and more. Each topic is represented as a JSON file containing a set of arguments and attack relations.

- **Work-Life Balance**
  - **Arguments:**
    - `a`: Completing work projects is crucial for satisfaction.
    - `b`: Taking breaks and going on vacation contribute to well-being.
    - `c`: Balancing work and personal projects is essential for happiness.
    - `d`: Consistent work efforts lead to professional success.
    - `e`: Extended vacations help in reducing stress and feeling relaxed.
    - `k`: Allocating time for personal interests out of work is the key to satisfaction.
  - **Attack Relations:**
    - Argument `e` attacks `d`.
    - Argument `d` attacks `b`.
    - Argument `b` attacks `a`.
    - Argument `c` attacks `b`.
    - Argument `a` attacks `k`.
- **Education vs. Work for Skills**
  - **Arguments:**
    - `a`: Investing in educational courses is the best way to gain professional skills.
    - `b`: Practical work experience is the best way to gain experience and grow in your career.
    - `c`: Networking is the best way to find opportunities for career growth.
    - `d`: Specializing courses contributes to career development.
    - `e`: Only I know what is best for my career development.
  - **Attack Relations:**
    - Argument `a` attacks `b`.
    - Argument `c` attacks `b`.
    - Argument `c` attacks `d`.
    - Argument `d` attacks `c`.
    - Argument `d` attacks `e`.
    - Argument `e` attacks `e`.
- **Work From Home vs. Office**
  - **Arguments:**
    - `a`: Working from home is more comfortable and allows for flexibility in your schedule.
    - `b`: Working from the office promotes collaboration and has fewer distractions.
    - `c`: Choosing when to do remote work and in-office work is the key to productivity.
    - `d`: Each person has their own preference for the number of days they go to the office.
    - `e`: Everyone in the team should be at the office on the same day.
  - **Attack Relations:**
    - Argument `a` attacks `b`.
    - Argument `b` attacks `a`.
    - Argument `b` attacks `c`.
    - Argument `c` attacks `d`.
    - Argument `d` attacks `e`.
    - Argument `e` attacks `c`.

2.1 Preferred Discussion Game

The implementation of the preferred discussion game includes a main class named `DiscussionGame`. The class represents a preferred discussion game where players take turns attacking and defending arguments until one of the winning conditions is met.

Class Explanation

The `DiscussionGame` class has the following attributes and methods:

- `arguments`: A dictionary containing arguments loaded from a JSON file.
- `attack_relations`: A list specifying attack relations between arguments.
- `claimed_argument`: The argument claimed by the proponent.
- `proponent_moves`: A list of arguments made by the proponent.
- `opponent_moves`: A list of arguments made by the opponent.

Methods

- `load_arguments(json_file)`: Loads arguments and attack relations from a specified JSON file.
- `get_valid_opponent_moves(attacked_argument)`: Returns valid opponent moves based on attack relations.
- `get_valid_proponent_moves(attacked_argument)`: Returns valid proponent moves based on attack relations.
- `play_round()`: Executes a round of the game, where the proponent and opponent make moves.
- `start_game()`: Initiates the game and continues until one of the winning conditions is met.

Game Flow

1. The game begins by creating an instance of `DiscussionGame` with a claimed argument and a JSON file containing arguments and attack relations. For example:

```
    claimed_argument = "e"
    json_file_path = "3_work_from_home_vs_office.json"
    game = DiscussionGame(json_file_path, claimed_argument)
```

2. The `start_game()` method is called to initiate the game loop.

```
    game.start_game()

```

3. The loop continues until one of the winning conditions is met. In each round, the proponent's argument is displayed, and valid opponent moves are presented.

```
    proponent_argument = self.proponent_moves[-1]
    valid_opponent_moves = self.get_valid_opponent_moves(proponent_argument)
        if not valid_opponent_moves:
            return True


```

4. The opponent selects an argument to attack the proponent's position.

```
        opponent_choice = input("Select an argument to attack the proponent's
    argument: ")
        self.opponent_moves.append(opponent_choice)

```

5. The game checks for winning conditions such as no valid opponent moves, no valid moves for the proponent, contradiction by the proponent, or reuse of an argument by the proponent. If a winning condition is met, the game ends, and the corresponding message is printed.

```
        valid_moves_for_response = self.get_valid_proponent_moves(opponent_choice)
        if not valid_moves_for_response:
            return True

        proponent_response = random.choice(valid_moves_for_response)
        self.proponent_moves.append(proponent_response)

        if proponent_response in self.opponent_moves:
            return True

        if opponent_choice in self.proponent_moves:
            return True

```

2.2 Argumentation framework

The implementation of the argumentation framework was made focusing on the *credulous semantic* for admissible, preferred and complete. Following the principle that the previous semantics are, by definition, equivalent; and that to be *credulously* accepted the argument must be contained in at least one extension of the argument semantics, the implementation had to be adapted to the possibility of resolving the equivalence without the need of testing for all possible labellings. The implementation follows various stages where different labellings are defined. If the inputed argument is found in an acceptable labelling, the result will be communicated immediately, regardless of the future calculations.

Nonetheless, the implementation has its limitations. Due to time constraints, the implementation does not have the most robust labelling function, and is not fully optimized. These strengths and limitations, alongside the nature of the system, will be discussed in this section

Running the script

The implementation utilizes the aforementioned JSON files containing the list of arguments and their attack relations. The script takes two arguments, the JSON file to be read and an argument to be checked in the selected semantics. For example:

```
    python3 ArgumentSolverCredulousPriority.py 1_work_life_balance.json a

```

If the arguments are not provided, the script will default to the file `1_work_life_balance.json` and argument `a`.

Implementation stages

Given that we are prioritizing returning a value for the *credulous preferred, admissible, and complete semantics* (CredPrefAdmCompl), the script implements stages in which this is checked. This is specially important due to the equivalence between these semantics, making it necessary to corroborate only one to demonstrate the others.

$$Cred_{adm}(a, F) = Cred_{pref}(a, F) = Cred_{comp}(a, F)$$

Before any argument can be corroborated, an argument map `argument_map` must be built. This is done in the shape of a dictionary, utilizing the arguments as keys and the value being a list of two lists: one containing the attacked arguments (children), and the attackers (parents). Maintaining a list of parents facilitates iterating through the arguments when assigning a label.

Previous to attempting any labelling, there is one exception that needs to be handled. Proving that an argument is not CredPrefAdmCompl usually requires attempting all combinations of labels, disregarding those that are not acceptable, and corroborating that the proposed argument is not classified as *in* in all of the remaining cases. However, there is a case where an argument is always not CredPrefAdmCompl. That is, when the argument is not in the argument map. This means that no

semantics may contain the given argument, and thus it is not found in CredPrefAdmCompl. If this is the case, the script will immediately return CredPrefAdmCompl as false.

In addition to the `argument_map`, an `argument_labelling` is created to contain the base of the following labellings. All arguments are instantiated with a value of *undetermined*, and redefined utilizing the function `proposed_argument_labelling` to include the *grounded semantics*. Both the `argument_map` and initial `argument_labelling` will not change throughout the run, as the first is immutable to match what arguments are defined in the JSON file, and the other will serve as a basis for all other labellings, given that he argumenst in the *grounded semantics* must appear as *in* in all proposed labellings. If the argument is found to be *in* with the *grounded semantics*, the script will return the run time and true values for CredPrefAdmCompl.

The implementation will then attempt to create different labellings containing the various arguments as *in* with the function `admissibility`. In order to combat the possibility to fall into a loop, the system implements a check to notice when changes are made, in addition to taking in the list parameter `unchageable_arguments` (defaulted to an empty list). These refer to the arguments that cannot be changed if the labelling is to hold. Consider a cyclical attack between argument a, b, and c, where each attack one other argument to form a cycle. You may decide to label a as *in*, thus labelling b as *out*, and continuously c as *in*. However, at this point you would be pushed to label a as *out*. If you continue to do so, this would lead to a loop. However, as we proposed a as *in* the implementation will remember this and notice that a was assigned *in* as in `unchageable_arguments`, and thus this labelling cannot hold.

In order to simplify the decision making of a labelling holding true or not, due to time constrains, both functions `admissibility` and `proposed_argument_labelling` return the labelling map and a true or false passing value. If the labelling cannot hold, the later one will be `false`.

After calculating the admissible labellings, the script will once again check for the the argument requested, given that it has not been found yet. Following, the implementation will building a list of arguments for the *preferred semantics*, checking one last time for the argument.

If after all the calculations the argument is not found, it is reported as to not be CredPrefAdmCompl, within the attempted labellings.

It is important to notice that the implementation will notify the result of the *credulous semantics* as soon as they are available, along with its run time, and continue to run the calculation to report on all the labellings available. However, as mentioned, this other calculations do not affect the response time of the *credulous semantics* and can be omitted by simply stopping the script.

Implementation limitations

The implementation has some large well known limitations, mainly no proper handling for parent assignation when the assignment of children has stagnated. This is better illustrated with an example. Consider the third argument map proposed by "Work From Home vs. Office". You might be able to tell that it is possible to assign b and d as *in*, with all remainder augments as *out*. With the current implementation, proposing b as *in* will result in such resolution, but proposing solely d as *in* will be marked as not passing. This is due to the implementation immediately failing the labelling after a cyclical relation of odd number of arguments is found (c, d, and e in this case), without looking at the parents of c and their possible labellings.

Additionally, a known bug occurs in `proposed_argument_labelling`, where the label assignment seems to stop without running the necessary loops. You may notice this when running

```
python3 ArgumentSolverCredulousPriority.py 1_work_life_balance.json a

```

```
argument_labelling map
{'a': 'Undefined', 'b': 'Undefined', 'c': 'In', 'd': 'Undefined', 'e': 'In', 'k':
'Undefined'}

```

It is important to notice that this does not affect the results, as the function `admissibility` will attempt the labellings regardless of this issue. However, it is a place for improvement in performance, as these arguments would not have to be tested at a later stage.

This bug was left in place for two main reasons: it achieve the labelling for the *grounded semantics*, and the run times presented this paper were achieved with it in place. Once again, time constraints has challenged the implementation.

However, a rudimentary implementation where `proposed_argument_labelling` is run repeatedly until no other argument can be assigned can be found in the comments. This same rudimentary adjustment can be found close to the end before the `Grounded labelling map` is printed. This was placed after the *credulous semantics* value is decided, so it has no effect on the run time of the calculation.

3 Experiments and results

For each Argumentation Framework provided in this project, an experiment was conducted to test the implementations of the argument solver and the preferred discussion games. In the following section we describe three different experiments in which the three distinct abstract argumentation frameworks are used as test beds for the two implementations. More specifically, the results of the project are detailed through experiments across three argumentation frameworks: Work-Life Balance, Education vs. Work Skills, and Work From Home vs. Office. These experiments used the preferred discussion game and an argument solver to test the credulous acceptance of arguments under admissible, preferred, and complete semantics. The outcomes are presented in Tables 1, 2, and 3, which show the acceptance status and runtime for each argument in the different frameworks.

3.1 Experiment 1

For experiment 1 we use Work-Life Balance argumentation framework. We are specifically interested in the arguments a and k. For the preferred discussion game we commence with a proponent claim a or k to be labeled **in** in a preferred labeling of this argumentation framework. The results of the discussion games are provided below.

```
--- Preferred Discussion Game 1.1  ---

Claimed Argument: a

Proponent: a
Completing work projects is crucial for satisfaction.

Valid opponent moves: ['b']
Select an argument to attack the proponent's argument: b
Taking breaks and going on vacation contribute to well-being.

Proponent: d
Consistent work efforts lead to professional success.

Valid opponent moves: ['e']
Select an argument to attack the proponent's argument: e
Extended vacations help in reducing stress and feeling relaxed.

Opponent wins! There are no valid moves for the proponent.

--- Preferred Discussion Game 1.2 ---

Claimed Argument: k

Proponent: k
Allocating time for personal interests out of work is the key to satisfaction.

Valid opponent moves: ['a']
Select an argument to attack the proponent's argument: a
Completing work projects is crucial for satisfaction.

Proponent: b
```

```
Taking breaks and going on vacation contribute to well-being.

Valid opponent moves: ['d', 'c']
Select an argument to attack the proponent's argument: d
Consistent work efforts lead to professional success.

Proponent: e
Extended vacations help in reducing stress and feeling relaxed.

Proponent wins! There are no valid opponent moves.
```

Consequently, the argument solver is used to answer the credulous decision problem under the three semantics, admissible, preferred, and complete for both argument a and k in the Work-life balance framework. The results of the argument solver implementation can be found in Table 1.

	Admissible	Complete	Preferred	Run-time Avg (seconds)
a	True	True	True	0.0004305
b	False	False	False	0.0002802
c	True	True	True	0.0002853
d	False	False	False	0.0002666
e	True	True	True	0.0001919
k	False	False	False	0.0002423

Table 1: Credulous Semantics of Argumentation Framework 1.

3.2 Experiment 2

For this experiment we use the Education vs. Work for skills argumentation framework. For the preferred discussion game we test the implementation specifically on the arguments c, d and e. Again the proponent start with the claim that one of these arguments is labeled **in** in a preferred labeling of this argumentation framework. The results of the discussion games are provided below.

```
--- Preferred Discussion Game 2.1 ---

Claimed Argument: c

Proponent: c
Networking is the best way to find opportunities for carreer growth.

Valid opponent moves: ['d']
Select an argument to attack the proponent's argument: d
Specializing courses contributes to career development.

Proponent: c
Networking is the best way to find opportunities for carreer growth.

Proponent wins! There are no valid opponent moves.

--- Preferred Discussion Game 2.2 ---

Claimed Argument: d

Proponent: d
Specializing courses contributes to career development.

Valid opponent moves: ['c']
Select an argument to attack the proponent's argument: c
```

```
Networking is the best way to find opportunities for carreer growth.

Proponent: d
Specializing courses contributes to career development.

Proponent wins! There are no valid opponent moves.

--- Preferred Discussion Game 2.3 ---

Claimed Argument: e

Proponent: e
Only I know what is best for my carreer development.

Valid opponent moves: ['e', 'd']
Select an argument to attack the proponent's argument: e
Only I know what is best for my carreer development.

Opponent wins! The proponent used an argument previously used by the opponent.

--- Preferred Discussion Game 2.4 ---

Claimed Argument: e

Proponent: e
Only I know what is best for my carreer development.

Valid opponent moves: ['e', 'd']
Select an argument to attack the proponent's argument: d
Specializing courses contributes to career development.

Proponent: c
Networking is the best way to find opportunities for carreer growth.

Proponent wins! There are no valid opponent moves.
```

For the argument solver we tested the implementation by trying to answer the credulous decision problem for admissible, preferred, and complete semantics for the arguments *c, d* and *a*. The results are shown in Table 2.

	Admissible	Complete	Preferred	Run-time Avg (seconds)
a	True	True	True	0.0001878
b	False	False	False	0.0002617
c	True	True	True	0.0002589
d	True	True	True	0.0002084
e	False	False	False	0.0002174

Table 2: Credulous Semantics of Argumentation Framework 2.

3.3 Experiment 3

For the third experiment we utilized the Work From Home vs. Office argumentation framework to test our implementations. For both the preferred discussion game and the argumentation solver we use the arguments *b, d* and *e*. The proponent will start with a claim that either one of these arguments is labeled *in* in a preferred labelling of this particular argumentation framework. The results of the preferred discussion games are provided below

```
--- Preferred Discussion Game 3.1 ---

Claimed Argument: b

Proponent: b
Working from the office promotes collaboration and has less distractions.

Valid opponent moves: ['a']
Select an argument to attack the proponent's argument: a
Working from home is more confortable and allows for flexibility in your schedule.

Proponent: b
Working from the office promotes collaboration and has less distractions.

Proponent wins! There are no valid opponent moves.

--- Preferred Discussion Game 3.2 ---

Claimed Argument: d

Proponent: d
Each person has their own preference for the amount of days they go to the office.

Valid opponent moves: ['c']
Select an argument to attack the proponent's argument: c
Choosing when to do remote work and in-office work is the key to productivity.

Proponent: e
Everyone in the team should be at the office on the same day.

Valid opponent moves: ['d']
Select an argument to attack the proponent's argument: d
Each person has their own preference for the amount of days they go to the office.

Opponent wins! The proponent contradicts itself.

--- Preferred Discussion Game 3.3 ---

Claimed Argument: e

Proponent: e
Everyone in the team should be at the office on the same day.

Valid opponent moves: ['d']
Select an argument to attack the proponent's argument: d
Each person has their own preference for the amount of days they go to the office.

Proponent: c
Choosing when to do remote work and in-office work is the key to productivity.

Valid opponent moves: ['e', 'b']
Select an argument to attack the proponent's argument: b
Working from the office promotes collaboration and has less distractions.

Proponent: a
Working from home is more confortable and allows for flexibility in your schedule.
```

```
Proponent wins! There are no valid opponent moves.
```

Consequently, the arguments were used to answer the credulous decision problem for admissible, preferred, and complete semantics for which the results are shown in Table 3.

	Admissible	Complete	Preferred	Run-time Avg (seconds)
a	True	True	True	0.0002252
b	True	True	True	0.0002254
c	False	False	False	0.0002272
d	True	True	True	0.0002310
e	False	False	False	0.0002310

Table 3: Credulous semantics of Argumentation Framework 3.

4 Discussion

The implementation of the Argumentation project showcased a robust system for handling abstract argumentation frameworks, with a particular emphasis on the preferred discussion game and credulous decision problems. The preferred discussion game effectively provided a dynamic and rule-driven interaction between the proponent and opponent, with a user-centered simulation of a debate scenario. The semantics implementation successfully addressed credulous decision problems, revealing the acceptability of arguments under admissible, preferred, and complete semantics. Acknowledging limitations such as challenges in parent assignation and a known bug demonstrated transparency in the discussion, opening avenues for future improvements.

The project's experimentation and results, conducted across three argumentation frameworks, demonstrated the practical application of the developed system. The discussion games effectively showcased different outcomes based on the claimed arguments, reflecting the detailed nature of argumentative interactions. The argument solver's results for credulous decision problems provided valuable insights into the acceptability of arguments, contributing to a deeper understanding of abstract argumentation semantics. Future explorations could dive deeper into other algorithms and variations for more efficient argumentation processing and applying these frameworks to different fields for practical insights into the generalizability of the algorithm.

5 Conclusion

In conclusion, this project has made a significant mark in using argument techniques for improving decision-making systems. It combines real experiments with practical coding work focusing on different topics. It uses discussion game model to create situations similar to real debates, which helps in testing different ways of arguing. A key part of this project is the development of an argument solver, an essential tool for figuring out which arguments hold and are accepted under various conditions - admissible, preferred, and complete. They give a detailed look at how decision-making happens in complex argumentation systems. Moreover, it is shown how useful it is to mix theoretical models with practical applications. Overall, the project highlights the successful integration of theoretical concepts into a functional system, while also emphasizing areas for refinement and future development.

References

[1] Ronald J Brachman. The basics of knowledge representation and reasoning. *AT&T technical journal*, 67(1):7–24, 1988.

[2] Phan Minh Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–357, 1995.

[3] Marcos Cramer and Yannick Spörl. The choice-preferred semantics for relevance-oriented acceptance of admissible sets of arguments. In Pietro Baroni, Christoph Benzmüller, and Y N. Wáng, editors, *Logic and Argumentation*, pages 94–111, Cham, 2021. Springer International Publishing.

[4] Hannes Strass and Johannes Peter Wallner. Analyzing the computational complexity of abstract dialectical frameworks via approximation fixpoint theory. *Artificial Intelligence*, 226:34–74, 2015.

[5] Atefeh Keshavarzi Zafarghandi, Rineke Verbrugge, and Bart Verheij. Discussion games for preferred semantics of abstract dialectical frameworks. In *Symbolic and Quantitative Approaches to Reasoning with Uncertainty: 15th European Conference, ECSQARU 2019, Belgrade, Serbia, September 18-20, 2019, Proceedings 15*, pages 62–73. Springer, 2019.